Implement Client Script & UI Policy (Incident)

Project Documentation phase :

- **Project Overview**

Project Module: Incident Management (ITSM)

Target Table: Incident (incident)

Objective: To automate form behavior, enforce data integrity during resolution, and enhance user experience through declarative UI Policies and responsive Client-side scripting.

Key Components Implemented:

UI Policy: Automatically enforces mandatory requirements for Close code and Close notes when an incident is marked as Resolved or Closed.

Client Script (OnChange): Triggers an immediate notification message when an agent selects Priority 1 - Critical, prompting proper major incident management protocols.

- **Client Script Details**

An OnChange client script was deployed to monitor changes made to the Priority field dynamically.

Name: Notify on Critical Priority Selection

Table: Incident (incident)

UI Type: All (Desktop & Mobile)

Type: onChange

Field Name: priority

Global: false (Scoped strictly to the incident table for best performance)

Source Code:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  // 1. Exit early if the form is loading or value hasn't changed
  if (isLoading || newValue === oldValue) {
    return;
  }

  // 2. Check if the new priority is Critical (Value '1')
  if (newValue == '1') {
    g_form.addInfoMessage('High Priority (Critical) Incident selected. Please ensure communication protocols and Major Incident processes are initiated.');
  } else {
    // Clear info messages if priority changes away from critical
    g_form.clearMessages();
```

```
  }
}
```

❖ **UI Policy Details**

To avoid writing custom scripts for simple field visibility and requirement adjustments, a declarative UI Policy was implemented.

Short Description: Make Close Notes and Close Code Mandatory on Resolution

Table: Incident (incident)

Conditions: State \rightarrow is one of \rightarrow Resolved, Closed

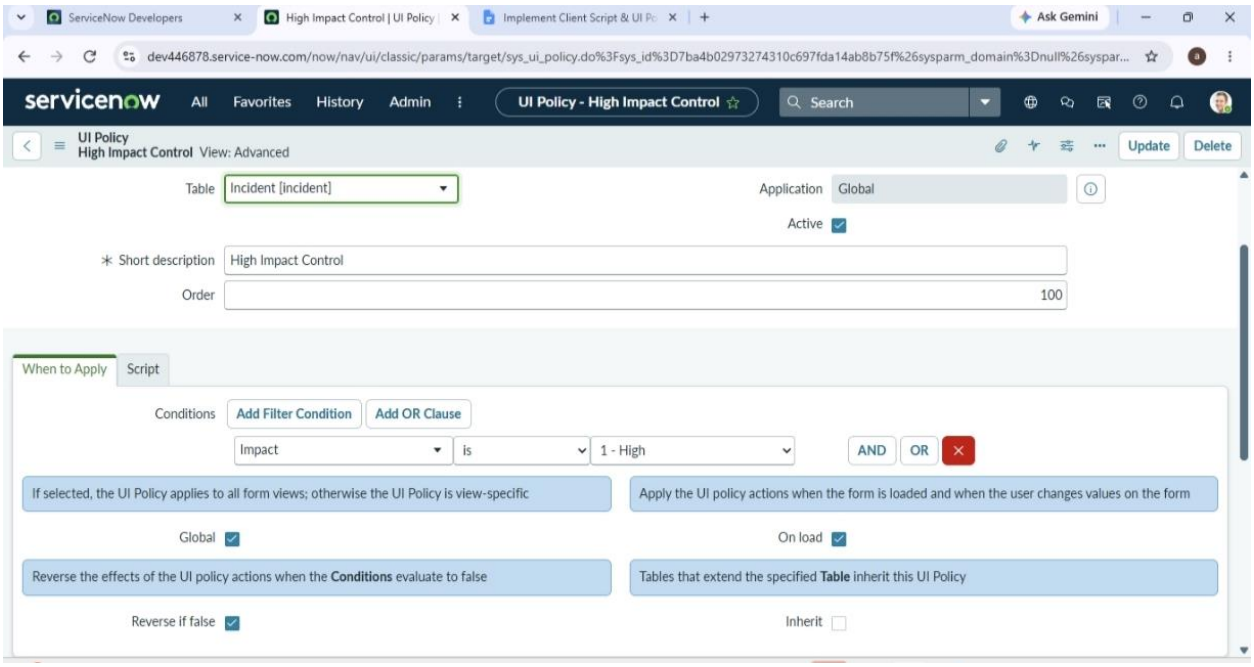
Global: true

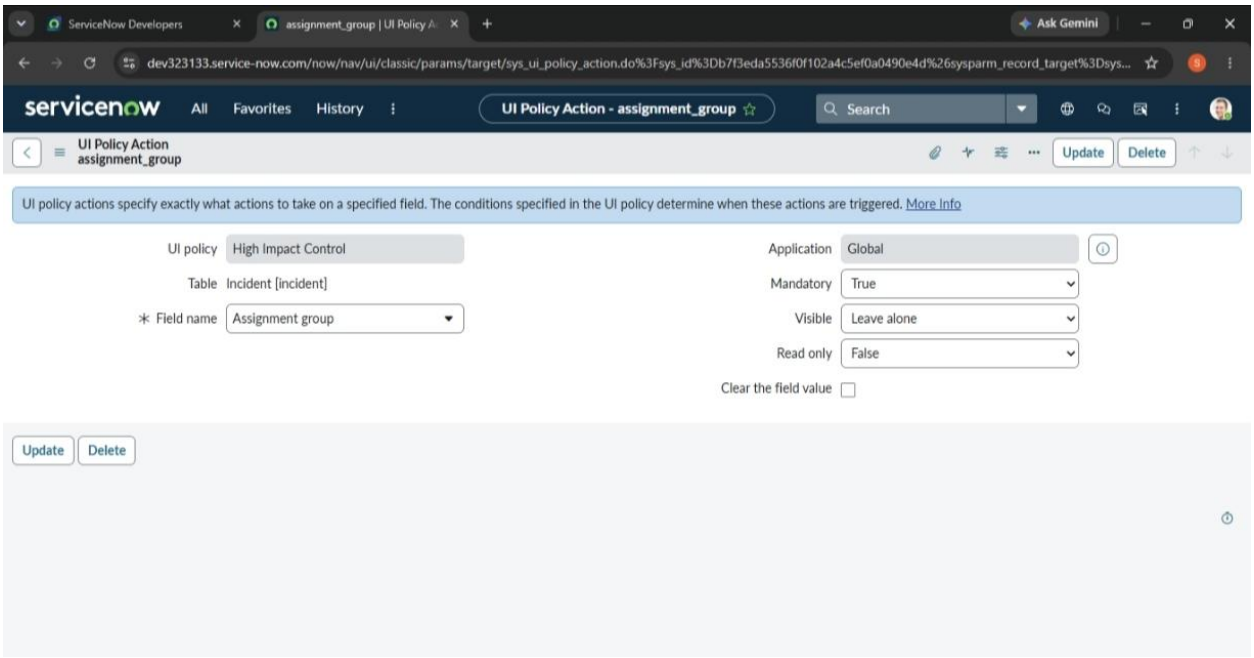
On Load: true (Evaluates state condition upon opening existing records)

Reverse if False: true (Resets fields back to non-mandatory when state changes back to Active/In Progress)

❖ **Screenshots**

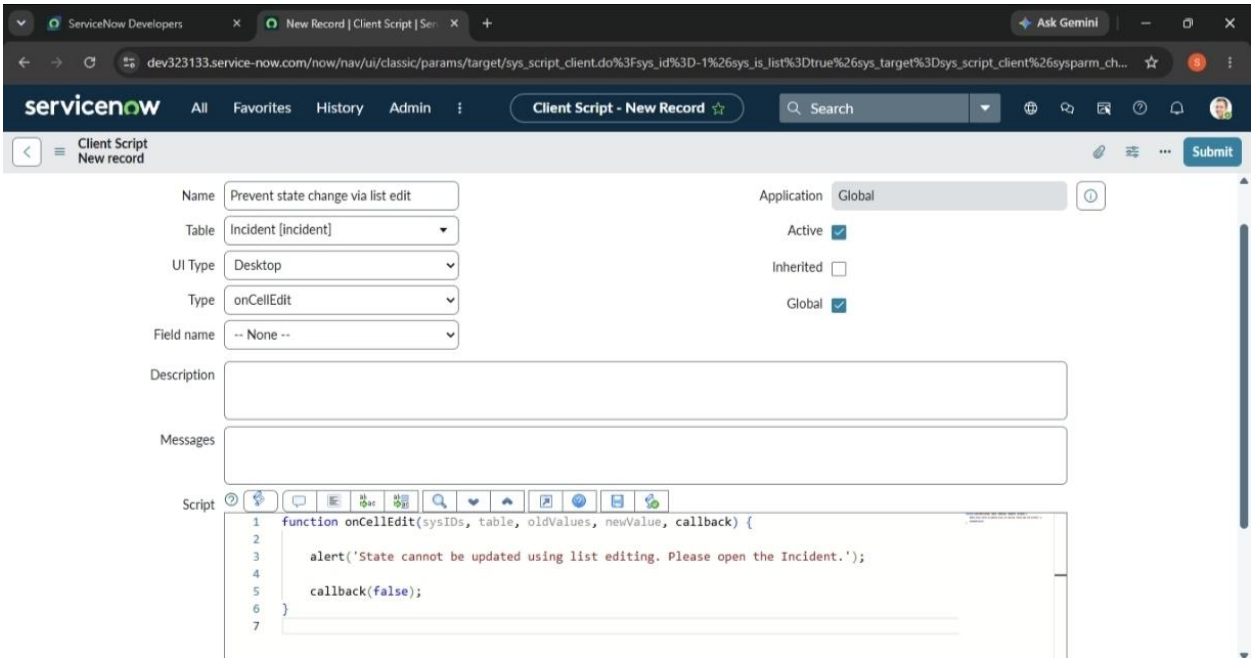
UI Policy On Incident :


ServiceNow Developers
High Impact Control | UI Policy
Implement Client Script & UI Po
Ask Gemini
dev446878.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy.do%3Fsys_id%3D7ba4b02973274310c697fda14ab8b75f%26sysparm_domain%3Dnull%26syspar...
servicenow
All
Favorites
History
Admin
UI Policy - High Impact Control
Search
UI Policy
High Impact Control View: Advanced
Update
Delete
Table
Incident [incident]
Application
Global
Active
Short description
High Impact Control
Order
100
When to Apply
Script
Conditions
Add Filter Condition
Add OR Clause
Impact
is
1 - High
AND
OR
If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific
Apply the UI policy actions when the form is loaded and when the user changes values on the form
Global
On load
Reverse the effects of the UI policy actions when the Conditions evaluate to false
Tables that extend the specified Table inherit this UI Policy
Reverse if false
Inherit

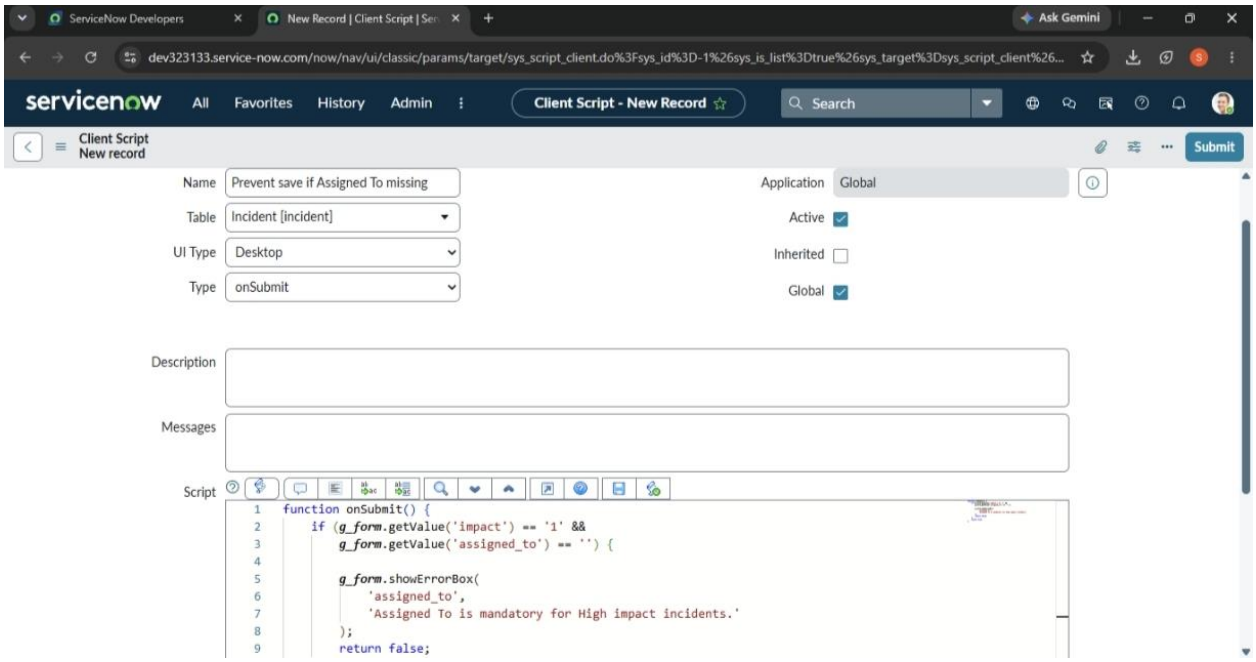

ServiceNow Developers
assignment_group | UI Policy A
Ask Gemini
dev323133.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy_action.do%3Fsys_id%3Db7f3eda5536f0f102a4c5ef0a0490e4d%26sysparm_record_target%3Dsys...
servicenow
All
Favorites
History
UI Policy Action - assignment_group
Search
UI Policy Action
assignment_group
Update
Delete
UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. More Info
UI policy
High Impact Control
Application
Global
Table
Incident [incident]
Mandatory
True
Field name
Assignment group
Visible
Leave alone
Read only
False
Clear the field value
Update
Delete

❖ **Client Script Record View**

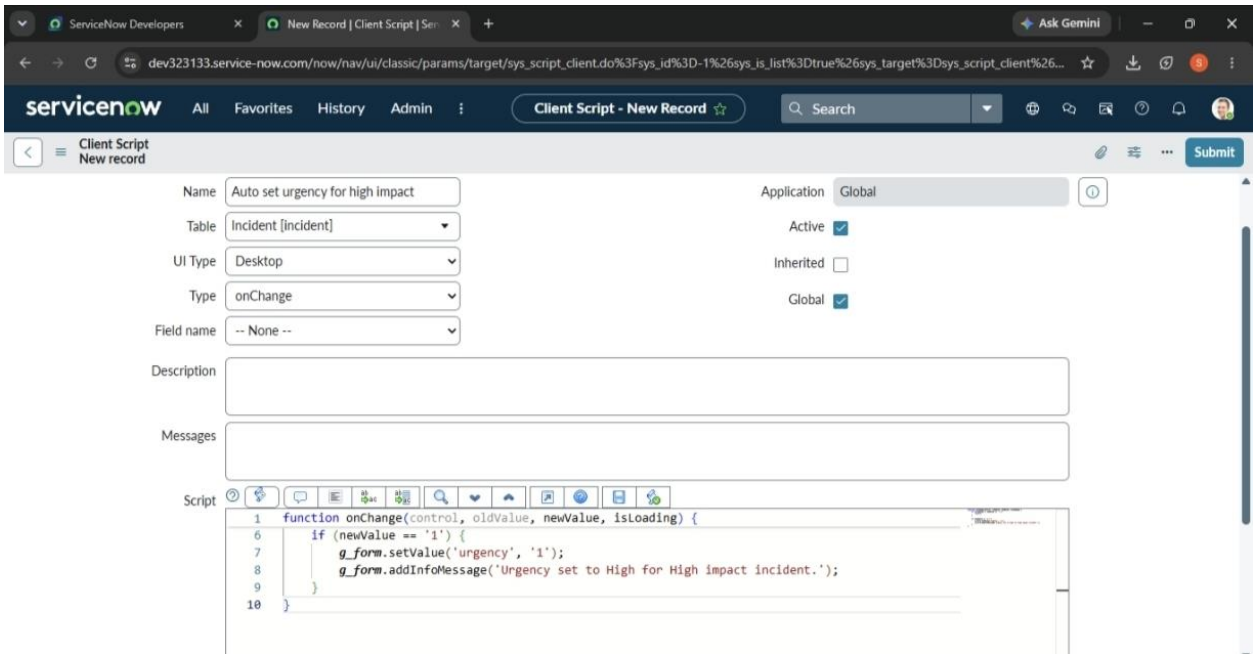
OncellEdit :



On submit :

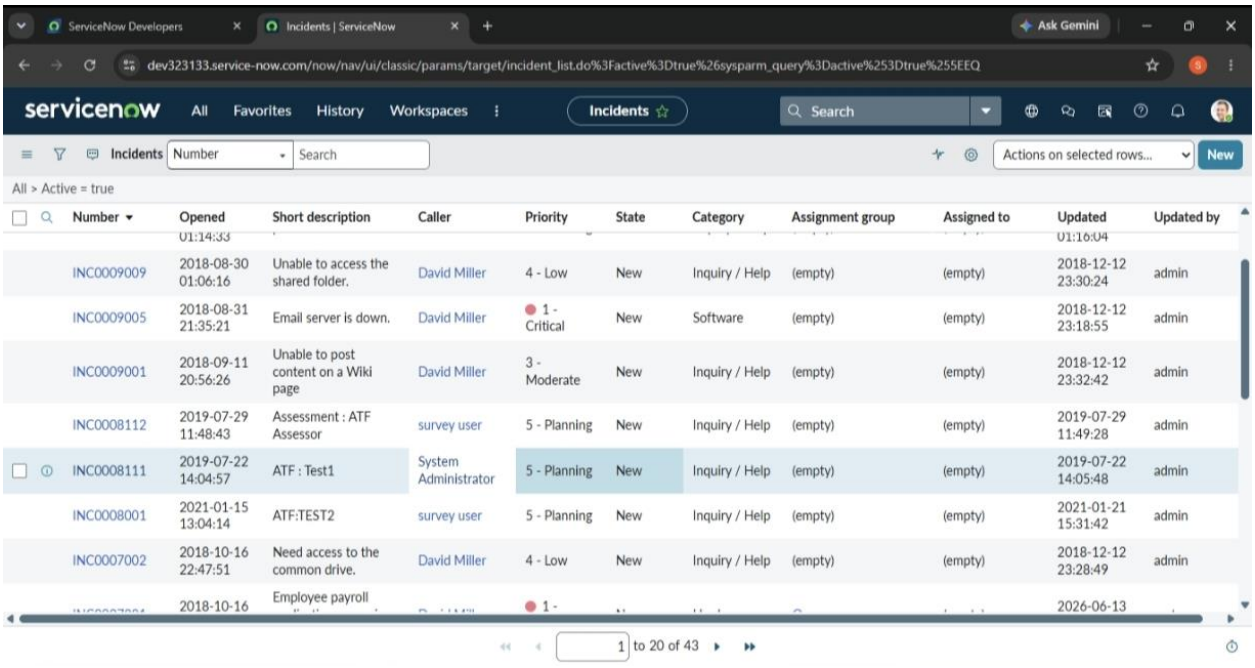


Onchange :



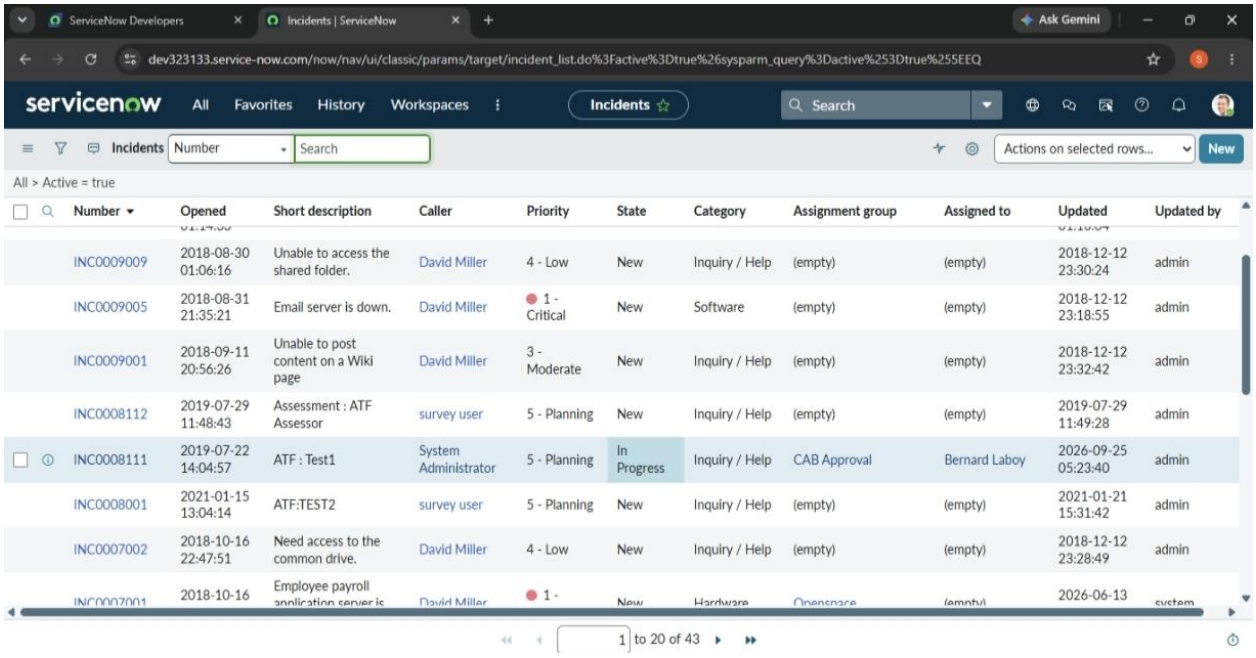
❖ Configuration Testing

Before



This shows the incident list prior to configuration. At this stage, standard data is present without any custom rules.

After



This displays the post-implementation state after testing the new configurations and policies. This confirms that the configured rules are functioning correctly

- **During the development and testing phase, the following challenges were encountered and resolved**

Problem 1: The OnChange script was firing unnecessarily and displaying unwanted alerts every time an existing incident record was loaded onto the screen.

Solution: Added a check for isLoading at the very beginning of the script function (if (isLoading || newValue === oldValue) { return; }) to block execution during initial form rendering.

Problem 2: When an agent changed an incident state back from Resolved to In Progress, the Close Notes remained mandatory, preventing form submission.

Solution: Enabled the Reverse if false checkbox in the UI Policy configuration, ensuring field requirements automatically revert when the condition is no longer met.

Problem 3: Direct DOM manipulation methods (like document.getElementById) were initially considered for changing field visibility, which is not best practice in ServiceNow.

Solution: Replaced custom DOM manipulation with native ServiceNow UI Policy actions and standard g_form API methods to ensure upgrade safety.

❖ Conclusion

The implementation of the Client Script and UI Policy on the Incident table successfully automated form behaviors and enhanced data governance. The UI Policy ensures that Close code and Close notes are strictly mandatory during ticket resolution, while the Client Script provides real-time alerts for critical incidents. By adhering to ServiceNow best practices, this project phase successfully improves agent workflows, data accuracy, and overall ITSM efficiency.